第5章 第7节 第一个RAG应用：游戏知识问答系统

- 来源: https://class.imooc.com/lesson/5273#mid=104919
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 第一个RAG应用：游戏知识问答系统
- 课节: 图文 1-1 游戏知识问答系统

1-1 游戏知识问答系统

第5章 第7节 第一个RAG应用：游戏知识问答系统

> [!TIP] 阅读指南 理论讲了这么多，该动手了。 这一节做一个真实的RAG应用——游戏知识问答助手。 《原神》玩家经常遇到各种问题：可莉的毕业武器是什么？胡桃的最优配装是什么？这个助手能从海量的游戏攻略、角色图鉴、玩家心得中找到最准确的答案。

7.1 《原神》角色知识库问答系统

做一个简易的《原神》角色知识库问答系统

数据资料

- 角色图鉴（角色介绍、属性、技能）
- 武器资料（武器类型、基础属性、特效）
- 圣遗物搭配（套装效果、推荐搭配）
- 玩家攻略（配队思路、输出手法）

功能

- 用户问： “胡桃用什么武器最好？”
- 系统检索相关资料
- AI基于真实数据给出专业回答

7.2 技术思路

回顾一下第2节讲的RAG流程：

```
准备数据 → 文档分块 → 向量化存储 → 用户提问 → 检索+生成

1
```

1

今天把这5步全部走一遍。

技术栈

- 向量数据库：Chroma

向量数据库

- Embedding模型：通义千问Embedding

Embedding模型

- 大模型：Qwen3.6-plus

大模型

- 编程语言：Python

编程语言

两阶段设计

在正式开始之前，先理解一个重要的概念：RAG系统分为离线构建和在线查询两个阶段。

这两个词来自软件系统的设计模式：

- 离线（Offline）：在用户访问之前就提前做好的准备工作，就像餐厅在开门前把食材都切好、调料都配好。
- 在线（Online）：用户实际访问时执行的操作，就像客人点菜后厨师现场烹饪。

在线（Online）

在RAG系统中：

- 离线程序：build_kb.py，提前把文档向量化存好，只运行一次

```
build_kb.py
```

离线程序

- 在线程序：query.py，每次用户提问时运行，负责检索和回答

```
query.py
```

在线程序

离线阶段（只运行一次）

> 读取原始文档 文档分块 向量化（调用Embedding API） 存储到向量数据库 输出：持久化的向量数据库

在线阶段（每次查询都运行）

> 加载已有的向量数据库 用户提问 检索相关文档 生成回答 输出：用户得到答案

- 加载已有的向量数据库
- 用户提问
- 检索相关文档
- 生成回答 输出：用户得到答案

生成回答

输出：用户得到答案

如果每次用户提问都重新向量化所有文档，完全没有必要，因为文档内容没变，向量也不会变。离线程序只需成功执行一次，把文本向量化存入数据库即可。

项目文件结构

```
game-rag/
├── data/                    # 原始数据
│   ├── 胡桃.txt
│   ├── 可莉.txt
│   ├── 钟离.txt
│   └── 甘雨.txt
├── build_kb.py              # 离线：构建知识库（运行一次）
├── query.py                 # 在线：问答系统（每次查询运行）
└── chroma_db/               # 向量数据库（自动生成）
    └── ...
```

[!TIP] 完整项目代码参考：samples/chapter5/game-rag/

7.3 离线构建

离线构建脚本负责读取文档、分块、向量化、存储。这个脚本只需要运行一次。

这个脚本只需要运行一次

[!TIP] 完整源码参考：samples/chapter5/game-rag/build_kb.py

按照RAG的流程，来看实现中的关键点。

配置区

```
API_KEY = os.getenv("DASHSCOPE_API_KEY", "sk-xxxxxxxxxxxx")
API_BASE = "https://dashscope.aliyuncs.com/compatible-mode/v1"

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "data")
DB_PATH = os.path.join(SCRIPT_DIR, "chroma_db")
```

- 环境变量优先：使用 os.getenv() 从环境变量读取API Key，避免硬编码泄露（强烈建议在生产环境使用这种方式）
- 路径变量：SCRIPT_DIR/DATA_DIR/DB_PATH 是目录和路径定义，不是API配置。它们属于脚本的"基础设施"，后面会被复用到多个地方

文档分块函数

```
def split_document(file_path):
    """读取文档并按空行分块"""
    with open(file_path, 'r', encoding='utf-8') as f:
        content = f.read()
    return [chunk.strip() for chunk in content.split('\n\n') if chunk.strip()]
```

- 按空行切分：split('\n\n') 利用数据文件的结构化特点

按空行切分

- 过滤空块：if chunk.strip() 避免空内容进入数据库

过滤空块

- 简单高效：对于结构化数据，简单的字符串切分就够了。如果是普通文档（如PDF、Word），可能需要使用NLP工具（spaCy、LangChain等）进行智能分句。但我们给出的示例数据已经用【】标记和空行预处理好了

创建客户端与Embedding函数

```
# 创建持久化客户端
client = chromadb.PersistentClient(path=DB_PATH)

qwen_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key=API_KEY,
    api_base=API_BASE,
    model_name="text-embedding-v2"
)

collection = client.create_collection(
    name="genshin_knowledge",
    embedding_function=qwen_ef,  # 注入Embedding函数
    metadata={"hnsw:space": "cosine"}  # 使用余弦相似度
)
```

- 持久化客户端：PersistentClient(path=DB_PATH) 告诉Chroma数据存到磁盘上chroma_db目录，而不是临时内存。下次启动时可以重新加载
- 依赖注入模式：Chroma不需要手动调用Embedding API，只需配置函数 qwen_ef
- 自动调用：在 collection.add() 时，Chroma会自动调用 qwen_ef 进行向量化

- 余弦相似度："hnsw:space": "cosine" 适合文本检索场景

批量加载文档

```
all_chunks, all_metadata, all_ids = [], [], []

txt_files = [f for f in os.listdir(DATA_DIR) if f.endswith('.txt')]
for filename in txt_files:
    character_name = filename.replace('.txt', '')
    file_path = os.path.join(DATA_DIR, filename)

    chunks = split_document(file_path)

    # 批量构建当前文件的所有数据
    all_chunks.extend(chunks)
    all_metadata.extend([
        {"character": character_name, "source": filename, "chunk_id": i}
        for i in range(len(chunks))
    ])
    all_ids.extend([f"{character_name}_{i}" for i in range(len(chunks))])
```

- 元数据设计：character、source、chunk_id 三个字段，方便检索后追溯
- 唯一ID：f"{character_name}_{i}" 确保每个文档块都有唯一标识
- 列表推导式：使用 extend() 配合列表推导，代码简洁

```
extend()
```

列表推导式

分批向量化存储

```
batch_size = 25  # Qwen API限制每批最多25个文本

for i in range(0, len(all_chunks), batch_size):
    collection.add(
        documents=all_chunks[i:i+batch_size],
        metadatas=all_metadata[i:i+batch_size],
        ids=all_ids[i:i+batch_size]
    )
```

- API限制：通义千问Embedding API一次最多处理25个文本，必须分批
- 直接传文本：Chroma会自动调用 qwen_ef 进行向量化，不需要手动调用

直接传文本

- 成本优化：44个文档块分2次API调用，之后就不再需要

成本优化

运行效果

```
$ python build_kb.py

开始构建知识库...
处理文件：钟离.txt
处理文件：胡桃.txt
处理文件：可莉.txt
处理文件：甘雨.txt

正在向量化 44 个文档块...
处理第 1 批（25 个文档块）...
处理第 2 批（19 个文档块）...

知识库构建完成！
数据库位置：./chroma_db
总文档块数：44
角色数量：4
```

7.4 在线查询

在线查询脚本负责加载知识库、检索、生成回答。这个脚本每次用户查询时运行。

这个脚本每次用户查询时运行

> [!TIP] 完整源码参考：samples/chapter5/game-rag/query.py

继续按照RAG的流程，来看实现中的关键点。

加载知识库

```
def load_knowledge_base():
    """加载已有的向量知识库"""
    client = chromadb.PersistentClient(path=DB_PATH)

    # 配置Embedding（必须与构建时一致）
    qwen_ef = embedding_functions.OpenAIEmbeddingFunction(
        api_key=API_KEY,
        api_base=API_BASE,
        model_name="text-embedding-v2"
    )

    # 获取集合（不是create，而是get）
    collection = client.get_collection(
        name="genshin_knowledge",
        embedding_function=qwen_ef
    )

    return collection
```

- get 而非 create：使用 get_collection() 而不是 create_collection()，加载已有数据
- 非常快：加载只需0.1秒，因为不需要向量化，只是读取元数据

非常快

- Embedding一致性：查询时使用的Embedding函数必须和构建时一致。因为查询时Chroma会用这个函数把问题向量化，然后和知识库中的向量计算相似度

```
def ask_question(collection, query):
    """基于RAG回答问题"""
    # 检索相关文档
    results = collection.query(
        query_texts=[query],
        n_results=3
    )
    retrieved_docs = results['documents'][0]

    # 构造上下文
    context = "\n\n".join([
        f"【参考资料{i+1}】\n{doc}"
        for i, doc in enumerate(retrieved_docs)
    ])
```

- 自动向量化：Chroma会自动调用 qwen_ef 把问题向量化

自动向量化

- Top-3检索：n_results=3 返回最相关的3个文档块

Top-3检索

构造Prompt

```
prompt = f"""你是一个《原神》游戏助手，请基于以下参考资料回答用户的问题。

参考资料：
{context}

用户问题：{query}

回答要求：
1. 基于参考资料回答，不要编造信息
2. 回答要专业但不失亲和力
3. 如果有多个选择，要说明优先级
4. 可以适当补充游戏常识

"""
```

- 角色定位：“你是一个《原神》游戏助手”，让AI进入角色

角色定位

- 明确指令：要求基于资料回答，不要编造，减少幻觉。这是RAG非常核心的意义——让大模型基于查询到的资料来回答，而不是直接凭记忆生成。

调用大模型

```
client = OpenAI(api_key=API_KEY, base_url=API_BASE)
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.7
)

return response.choices[0].message.content
```

- temperature=0.7：适度创造性，既不僵化也不过分发散
- 单轮对话：每次都是新对话，没有历史记忆（可扩展）

单轮对话

主程序：交互式问答

```
def main():
    # 加载知识库（只加载一次）
    collection = load_knowledge_base()

    # 交互式问答
    while True:
        query = input("你：").strip()
        if query in ['退出', 'quit', 'exit']:
            break

        if not query:
            continue

        answer = ask_question(collection, query)
        print(f"\n助手：{answer}\n")
```

- 只加载一次：知识库在循环外加载，多次查询复用
- 空输入过滤：if not query: continue 避免空查询

空输入过滤

- 退出机制：支持中英文多种退出方式

退出机制

```
$ python query.py

==================================================
《原神》知识问答助手
==================================================

正在加载知识库...
知识库加载完成

开始对话（输入'退出'结束）

你：胡桃用什么武器最好？

助手：胡桃的最佳武器推荐如下：

毕业武器（首选）
**护摩之杖**（五星）
- 基础攻击：608
- 副属性：暴击伤害66.2%
- 特效：生命值转化攻击力，完美契合胡桃机制

替代选择
**龙吟**（五星）适合暴伤型配装
**决斗之枪**（四星）平民神器，性价比高

建议：有护摩优先护摩，没有可以用龙吟或决斗之枪。
```

测试：超出知识库范围

```
问：明天会下雨吗？
答：哎呀，旅行者，关于"明天会不会下雨"这个问题，我得老实告诉你—我的参考资料里可没有天气预报功能哦！ 😅。不过既然你提到了"雨"，咱们倒是可以聊聊《原神》里的角色配队！比如在甘雨的冻结队中，行秋或莫娜就能提供稳定的水元素附着，配合甘雨的冰箭打出"冻结"反应，

1
2
```

7.5 下一节预告

到这里，已经用不到200行代码实现了一个能跑的RAG应用。但现实中的文档不会这么规整——100页的PDF、扫描版、双栏排版、表格跨页……该怎么切？切得太大，检索不精准；切得太小，语义不完整。下一节来看文档分块这门艺术，看看它是如何决定RAG系统的生死。

7.6 ■ 学点英语

中文	English	音标	说明
知识库	Knowledge Base	/ˈnɑːlɪdʒ beɪs/	存储结构化/非结构化文档的系统，RAG检索的数据来源
元数据	Metadata	/ˈmetədeɪtə/	描述数据的数据，如文档来源、类型、标签、时间等
语义相关度	Semantic Relevance	/sɪˈmæntɪk ˈreləvəns/	检索结果与用户查询之间的语义匹配程度
上下文质量	Context Quality	/ˈkɑːntekst ˈkwɑːləti/	传给AI的检索结果的相关性和完整性度量